Computational Suitability Statement (CSS)

Scientific Overview

Mobile robots today often navigate obstacle-free shortest paths to coordinates, but real-world instructions are far richer: directional, landmark-based, constraint-following, and referential. Current Vision-Language Navigation models fail to follow such fine-grained instructions and do not generalize across environments [3,4]. Inspired by recent success leveraging crowd-sourced street-view imagery for navigation [1,2], we propose training a steerable navigation model on world-scale street-view data by fine-tuning vision-language models (3B–32B) on millions of image-instruction pairs.

Our approach has three phases: (1) large-scale data acquisition and processing, processing street-view imagery for trajectory information and leveraging VLMs for labeling, (2) supervised fine-tuning of VLM models for trajectory prediction conditioned on language instructions, and (3) real-robot evaluation on a mobile manipulation platform. Phases 1 and 2 are GPU-intensive, requiring sustained multi-GPU training and inference over hundreds of GPU-hours per experiment. We are targeting CoRL 2026 (May deadline).

Prior Experience on GPU-based Systems

Our team has extensive experience working with GPU-based compute infrastructure, having completed a range of projects, both in industry and in academia. We regularly use our lab's SLURM-managed Juno cluster, housed within the Stanford SC Compute Cluster, for training and evaluation. We also have significant experience provisioning and managing GPU instances on Google Cloud Platform, including H100 VMs for VLM fine-tuning.

Our team includes a member with industry experience as an ML engineer specializing in scaling distributed training workloads, as well as researchers with direct experience training large-scale perception models [1]. The predecessor to this project, MapItAnywhere [1], involved training a large-scale environment prediction model on a multi-node SLURM-managed cluster at another institution, giving us direct experience with distributed multi-GPU training workflows.

Through these experiences, we have developed standard protocols and guidelines for multi-GPU jobs, including monitoring GPU utilization, memory saturation, and throughput during training to ensure we are maximizing hardware usage. We routinely profile jobs early in a run and adjust batch sizes, gradient accumulation, and parallelism strategies before committing to full-scale experiments.

Scaling Studies

The proposed work builds on our previously successful project, MapItAnywhere [1], which demonstrated that scaling training data to world-scale street-view imagery significantly improved generalization of a BEV map prediction model beyond what was achievable with conventional, geographically limited datasets. We use the same data source for this project and expect similar scaling trends for language-conditioned navigation.

This is consistent with broader findings in the robotics scaling literature. Sartor and Thompson [5] conducted a comprehensive analysis of scaling laws for robot foundation models and found that “the performance of robotic models improves with increased resources, following a power-law relationship”.

We plan to conduct scaling studies during the project, measuring model performance as a function of training data size (10K, 100K, 1M) and model size (3B to 32B parameters), to study these trends for our navigation task.

Codes and Toolchains

Our training pipeline is built on well-established, actively maintained machine learning frameworks. For supervised fine-tuning of our navigation model, we use HuggingFace TRL (https://github.com/huggingface/trl) with PyTorch, to finetune on the Qwen2.5-VL-3B vision-language model. We use LoRA for parameter-efficient fine-tuning, with a rank empirically tuned to balance training efficiency and downstream task performance. These frameworks include built-in support for multi-GPU training, mixed-precision (bf16), flash attention, and gradient checkpointing.

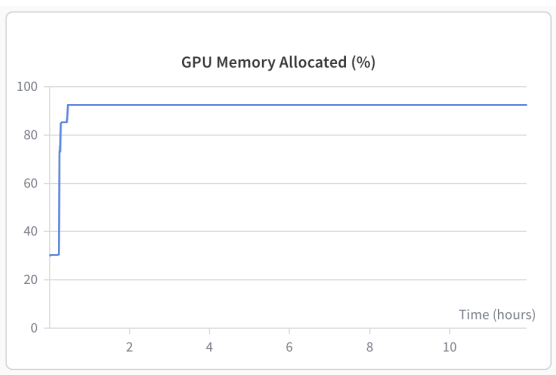
We plan to additionally explore VeRL (https://github.com/volcengine/verl) for RL-based fine-tuning, and may extend to the Qwen 3.5 model family (https://github.com/QwenLM/Qwen3.5) that includes smaller variants optimized for faster inference on robot hardware.

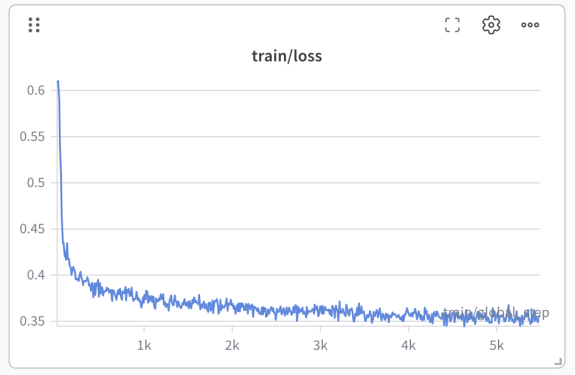
Job Profile

Our scaled-up training jobs will utilize 2–8 NVIDIA H100 GPUs on 1–2 nodes, with individual experiments running for 12–36 hours. We typically run 1–2 concurrent jobs, scaling to 4–8 concurrent jobs during peak experimentation periods (e.g., pre-deadline ablation studies). GPU utilization is maintained at 80–100% with minimal CPU demands (~3%) and typical RAM usage of 20–64 GB. Training data will be preloaded to node-local storage, with minimal I/O during training. Our pipeline does checkpointing (~10 GB per checkpoint) automatically, enabling reliable job pause and restart.

Computational Readiness and Performance

We have confirmed our training implementation uses GPUs effectively. Attached profiling from an initial small-scale study (37K samples, batch size 8) on a single A5000 GPU shows consistent ~90% saturation of the available 24 GB VRAM. We use LoRA with an empirically tuned rank to balance efficiency and task performance. We expect a modest MFU typical of LoRA fine-tuning, where only a small fraction of parameters are trainable.
For this project, we will use our data pipeline to generate approximately 2M training samples from street-view imagery, a scale necessary for effective generalization across diverse environments and instruction types. Training on this full dataset requires significantly larger batch sizes and multi-GPU training that our current lab resources cannot support. Marlowe's extensive H100 infrastructure would provide the compute capacity needed to train at this scale.





Technical Requirements & Feasibility:

All software dependencies are open-source and pip-installable. Training relies on HuggingFace TRL and PyTorch, with standard dependencies including CUDA 12 and Hugging Face Transformers. Model weights (Qwen2.5-VL-3B) are publicly available on HuggingFace Hub. No proprietary software, special hardware peripherals, or non-standard kernel configurations are required.

Storage & Data:

Our storage needs primarily consist of scratch space for large-scale image data, intermediate processing, and model checkpoints. Training data is sourced from the publicly available Mapillary street-view image API (~5TB), downloaded directly to cluster storage or transferred via rsync. Intermediate artifacts generated from a curated set of raw data include processed image-instruction pairs, point clouds, trajectory overlays, and VLM-generated instruction labels (~20TB). We checkpoint models regularly during training (~10 GB per checkpoint using LoRA adapters), retaining only the most recent and best-performing checkpoints while cleaning up unused data after runs. We estimate an overall scratch storage requirement of approximately 50 TB.

Justification for Marlowe

We are requesting for Medium Project Access with Marlowe due to the inadequacy of available GPUs through standard lab or cloud resources (Google CoLab, lab servers, GCP). Our project leverages internet-scale data to train navigation models, requiring sustained multi-GPU compute for training, much more compute needed than standard sources. Fine-tuning 3B-7B parameter models on our large-scale dataset demands 4-8 GPUs per experiment over 12–36 hours, with a total estimated need of 10,000 GPU-hours over 3 months.

Our lab cluster (Juno) is shared across multiple projects and is in particularly high demand during conference deadline periods. We have applied for GCP research credits, but anticipate that the allocation will be insufficient to support the large-scale training and scaling studies required for this project. Provisioning equivalent GPU-hours through commercial cloud providers is cost-prohibitive at our scale. Marlowe's extensive H100 GPU resources would provide the reliable, high-compute GPU access (multi-day runtime) needed to meet our timeline.

Impacts and Acknowledgements

This project addresses a key barrier in robot navigation: the scarcity of large-scale, diverse training data. By leveraging already-available internet-scale street-view imagery, our approach unlocks a virtually unlimited data source for training navigation models that can follow diverse natural language instructions anywhere. We plan to publish at CoRL 2026, submit follow-up works to subsequent venues and open-source all code, models, and data, providing the robotics community with a steerable navigation foundation model, a scalable data pipeline, and an evaluation benchmark for language-conditioned navigation.

References

[1] Ho, C., Zou, J., Alama, O., Jagadesh Kumar, S. M., Chiang, B., Gupta, T., Wang, C., Keetha, N., Sycara, K., & Scherer, S. (2024). Map it anywhere: Empowering BEV map prediction using large-scale public datasets. In Advances in Neural Information Processing Systems (NeurIPS), Dataset and Benchmark Track.

[2] Sarlin, P. and DeTone, D. and Yang, T. and Avetisyan, A. and Straub, J. and Malisiewicz, T. and Rota Bulo, S. and Newcombe, R. and Kontschieder, P. and Balntas, V. (2023). OrienterNet: Visual Localization in 2D Public Maps with Neural Matching. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

[3] Guaman Castro, M., Rajagopal, S., Gorbatov, D., Schmittle, M., Baijal, R., Zhang, O., Scalise, R., Talia, S., Romig, E., de Melo, C., Boots, B., & Gupta, A. (2025). VAMOS: A Hierarchical Vision-Language-Action Model for Capability-Modulated and Steerable Navigation. arXiv:2510.20818.

[4] Hirose, N., Glossop, C., Shah, D., & Levine, S. (2025). OmniVLA: An Omni-Modal Vision-Language-Action Model for Robot Navigation. arXiv.

[5] Sartor, S., & Thompson, N. (2024). Neural scaling laws in robotics. arXiv preprint arXiv:2405.14005.